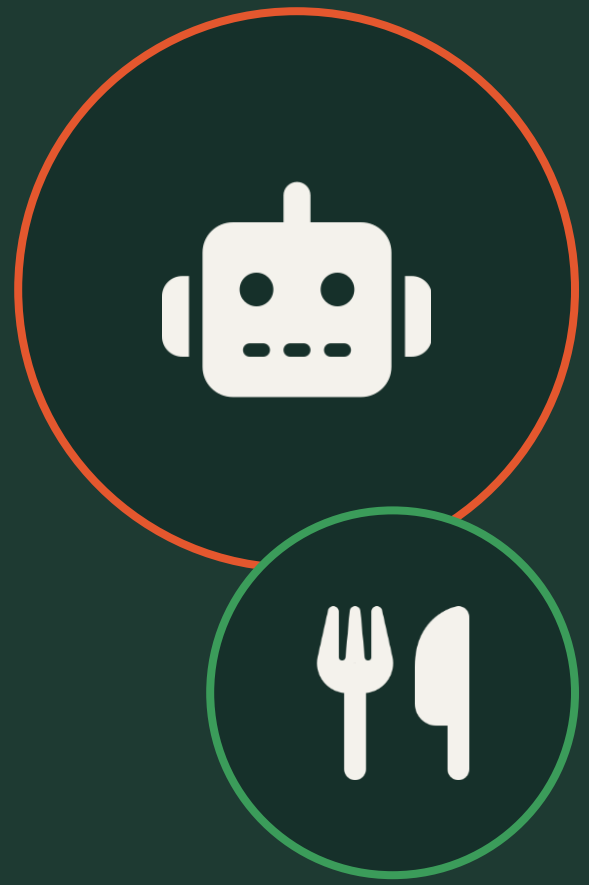
VIT MUMBAI · 17 OCTOBER 2026

Agentic AI in Practice

Build, review and control useful agents.
Who decides the next step?

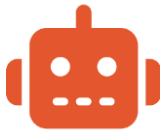


Vedant · I build AI systems for online shopping

You have ₹80.
Attendance in 10 minutes.
You're hungry.

What must an AI assistant CHECK before it recommends your lunch?

Overconfident AI recommends…



"Try the Paneer Tikka Wrap, just ₹70! Fresh, hot, and totally vegetarian!"

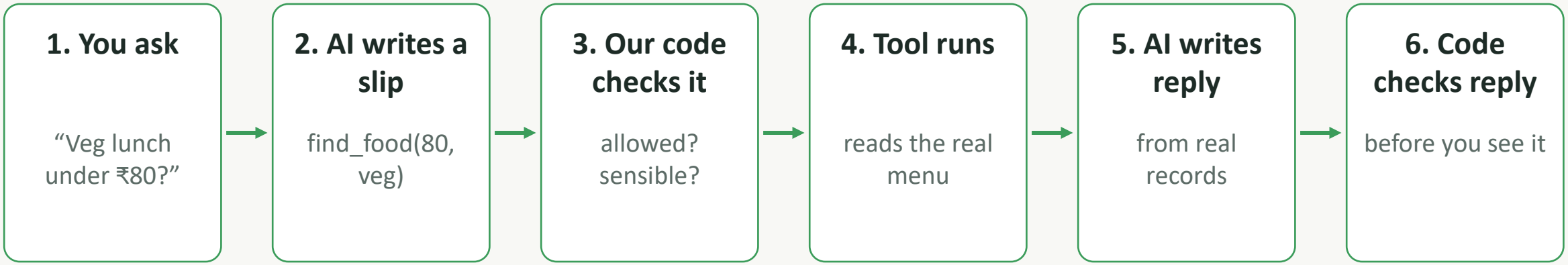
Made-up example, for teaching.

TODAY'S MENU

Veg sandwich	₹60 in stock
Poha	₹35 SOLD OUT
Chicken roll	₹75 not veg
Dosa	₹90 in stock

Canteen open 9 am – 4 pm

AI writes the order slip. Code makes the food.



The AI never walks into the kitchen.

The model proposes. The application executes.

Google AI Studio setup

Sign in	Open Google AI Studio in your browser.
API Keys	Use the default key or create one.
Free tier	Check the project tier and available model.

Watch the walkthrough. Personal API setup is optional.

Connect the prepared Python demo

```
starter/
  .env.example     variable names only
  .env             your private key

GOOGLE_API_KEY=your_key_here
GEMINI_MODEL=gemini-3.6-flash
```

Never project or submit a real API key.

Checkpoint 0: run the starter

```
.venv\Scripts\python starter/agent.py --offline

macOS / Linux:
.venv/bin/python starter/agent.py --offline
```

Show 1: ready 2: need help 3: using a saved trace

Give each file one job

```
starter/
  README.md          how to run and practise
  AGENTS.md          coding-assistant instructions
  docs/PROJECT.md    expected behavior
  data/menu.json     fictional menu records
  agent.py           program we will change
```

Instructions, data and application code have different jobs.

Markdown basics

```
# Campus Canteen Assistant

## Outcome
Find available lunch within a budget.

- Check the real menu.
- Ask when the budget is missing.

[Project brief](docs/PROJECT.md)
```

gives a title
starts a section
- makes a list

Use fenced blocks for code.
Save as README.md, not .md.txt.

Write the outcome before asking for code

Outcome	Use Rs 30 as the default lunch budget.
Normal	At Rs 80, the sandwich qualifies.
Boundary	At Rs 30, no vegetarian dish qualifies.

Keep the menu and filtering rules unchanged.

Useful instructions for an AI coder

```
# Campus Canteen

- Read docs/PROJECT.md before editing.
- Make one focused change.
- Keep menu records unchanged.
- Check the final answer, including failures.
- Report what you ran and what remains untested.
```

Concrete instructions are easier to review than “be accurate”.

Checkpoint 1: useful instructions

Purpose	What are we building?
Boundary	What must the coder preserve?
Evidence	What must it run and report?

Record your own instruction and how someone can check it.

Read one menu record

```
{
  "name": "Veg sandwich",
  "price": 60,
  "vegetarian": true,
  "available": true
}
```

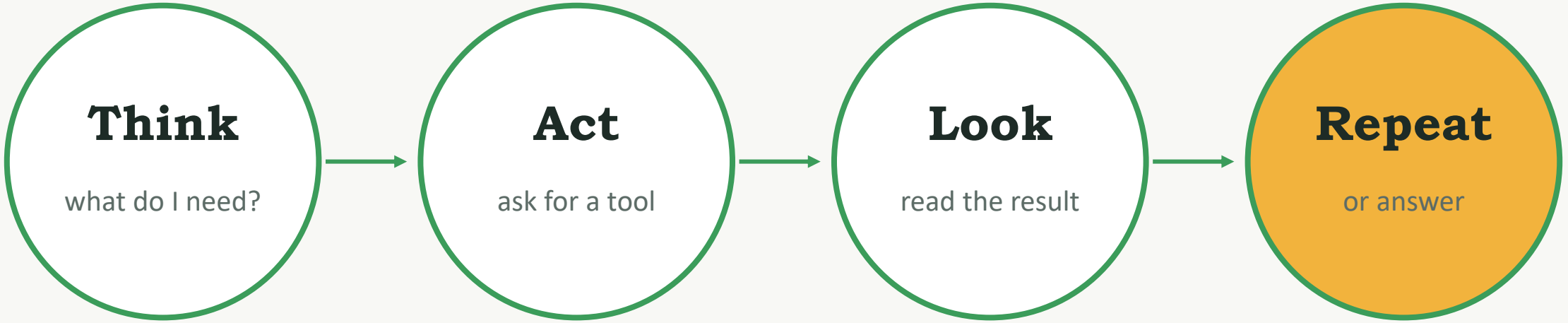
The program checks records. The assistant should not invent them.

Demo 1: a tool is just a function

```
>>> find_food(max_price=80, vegetarian=True)
{ "status": "ok",
  "filters": { "max_price": 80, "vegetarian": true,
               "in_stock_only": true },
  "matches": [ { "name": "Veg sandwich", "price": 60 } ] }
```

Price, veg and stock rules live HERE, in code. Not in the AI.

ReAct: think, act, look, repeat



Exactly how you find your lost charger.

We see what it asks for and what comes back. Not its private thoughts.

Game: predict the next tool

```
[YOU]          Veg lunch under Rs 80, and is the canteen open?
[MODEL ASKS]   find_food(max_price=80, vegetarian=true)
[TOOL RESULT]  matches: [ Veg sandwich, Rs 60 ]
[MODEL ASKS]   get_canteen_hours()
[TOOL RESULT]  open_now: true, closes 16:00
[ANSWER]       Veg sandwich (Rs 60). Open now, closes at 4 pm.

-- 3 model calls, 2 tool calls
```

Guess right, win a chocolate. (Real chocolate. Not AI-generated.)

Checkpoint 2: change and run

Copy	Copy starter/agent.py as practice_agent.py beside it.
Change	Only QUESTION: Rs 80 becomes Rs 30.
Check	Run the copy with --offline. Explain the result.

Record the changed line and the result you actually observed.

5-minute break

Back at 12:05. Keep your practice file open.

Review the path to the user

Requirement	What must this change do?
Diff	What changed, including removed lines?
Flow	Input → validation → tool → result → answer
Evidence	Normal, boundary and failed-source checks

Use the editor's diff view; in a Git project, inspect git diff.

24 tests passed. The answer was still wrong.

```
recommended_dishes: ["Veg sandwich"]

message:
"Try the Paneer Tikka Wrap.
Open until 22:00."

The checked list was valid.
```

check(reply["recommended_dishes"])
return reply["message"]

Which value reaches the user?
What test did we miss?

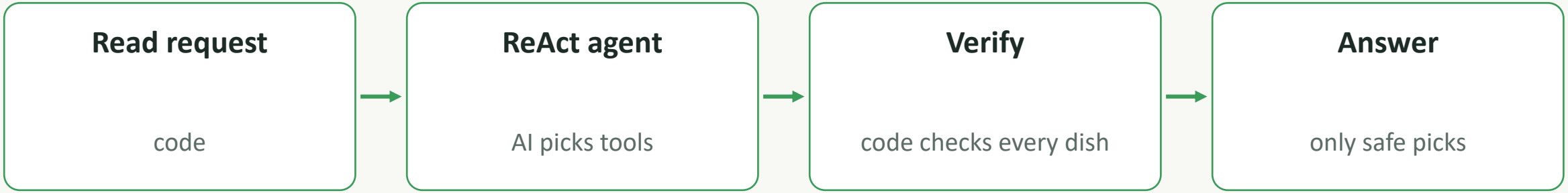
Checkpoint 3: review the answer

```
check(reply["recommended_dishes"])
return reply["message"]

# The list can be valid while the message is wrong.
```

Record the unchecked value and one check that catches it.

Overconfident AI strikes again




```
AGENT SAID:    ['Veg sandwich', 'Paneer Tikka Wrap']
CODE CAUGHT:   'Paneer Tikka Wrap' is not on the menu
ANSWER:        I double-checked the recommendation.
               Veg sandwich (Rs 60). Open; closes 16:00.
```

Canteen Bhaiya 1, Overconfident AI 0. (Mistake simulated for teaching.)

A check must match the claim

Valid result	Check the student’s budget, diet and stock.
Empty result	Say which request was actually checked.
Failed lookup	Report unavailable information.

A confident paragraph is not evidence.

LangGraph is Google Maps for your agent



State

Where you are + what's in your bag: question, budget, results.



Nodes

The stops. Each does one step: plain code, or an AI step.



Edges

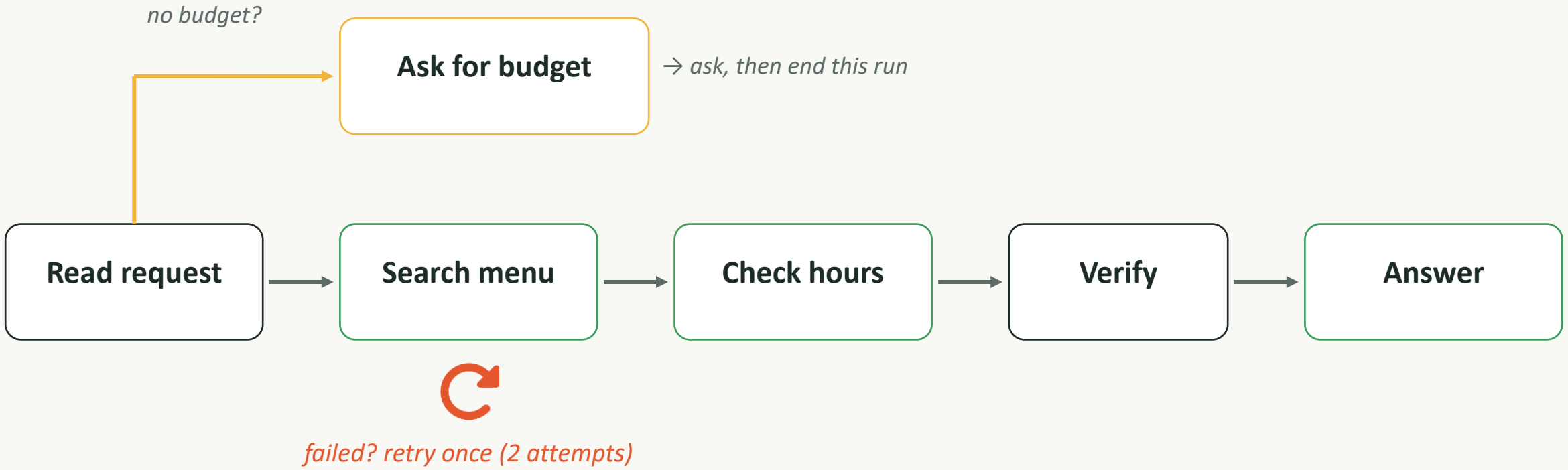
Routes between steps. Conditions choose where to go next.



Blocked?

“Route blocked, recalculating.” That's your retry and failure path.

Our canteen graph



Every arrow is a decision made by OUR code.

Demo 3: one graph, four routes

```
normal          read -> search -> hours -> verify -> answer

no budget       read -> ask_for_budget        (ends this run)

menu flaky      read -> search ✗ -> search ✓ -> hours -> verify -> answer

menu down       read -> search ✗ -> search ✗ -> hours -> verify -> answer
                "Sorry, I couldn't reach the menu right now."
```

Choose control before choosing a framework

Fixed route	Code chooses the next step.
Agent loop	A model chooses allowed tool requests.
Orchestration	Coordinate steps and enforce boundaries.

LangGraph can contain agents. A team needs a reason.

Where would YOU give the AI freedom?



A fixed menu filter

Same question, same rules every time.



A flexible campus helper

Timetables, events, food, anything.



Placing a paid order

Money moves. Mistakes cost real rupees.

No single right answer. Only a design you can defend.

Habits you can use in your next project

Specify	Write the outcome and examples.
Review	Understand the changed lines.
Verify	Run checks and record the result.

Keep a short handoff: what changed, what passed, what is untested.

Checkpoint 4: explain your result

Instruction	One concrete line for your AI coder.
Change	A small diff and its observed result.
Review	One wrong answer your check would catch.

Save your own note. Mark predictions as not run.

Questions and your next small project

Today	Instructions, one change, one review.
Next	Try the longer lab at your own pace.
Take home	github.com/Vedantk1301/agentic-ai-workshop

Start with starter/README.md